Meet Up (การนัดพบ)

ราชอาณาจักรมีเส้นรถไฟเพียงสายเดียวเป็นเส้นตรงยาวเชื่อมสถานีเรียงต่อกัน โดยเวลาการเดินทางระหว่างสถานีติดกันเท่ากัน แต่ละสถานีถูกเก็บในโครงสร้างข้อมูลแบบ linked list ดังนี้

```
struct station {
    station* left  = nullptr;  // พอยน์เตอร์ไปยังสถานีทางซ้ายมือ
    station* right = nullptr;  // พอยน์เตอร์ไปยังสถานีทางขวามือ
    int left_time  = 0;        // เวลาเดินทางไปสถานีทางซ้ายมือ
    int right_time = 0;        // เวลาเดินทางไปสถานีทางขวาขวา
};
```

สำหรับสถานีที่อยู่ซ้ายสุดจะมีค่า `left` เป็น `nullptr` และ `left_time` เป็น 0 ส่วนสถานีที่อยู่ขวาสุดจะมีค่า `right` เป็น `nullptr` และ `right_time` เป็น 0

ฟรีเรนอยู่ที่สถานีที่ถูกชี้ด้วยพอยน์เตอร์ `s` และฮิมเมลอยู่ที่สถานีที่ถูกชี้ด้วยพอยน์เตอร์ `t` ทั้งสองสามารถนั่งรถไฟไปทางซ้ายหรือขวาก็ได้และต้องพบกันที่สถานีเท่านั้น หากคนใดมาถึงก่อน ต้องรออีกฝ่ายให้ถึงก่อนจึงจะถือว่าเจอกัน จงหาว่าเวลาที่น้อยที่สุดที่ทั้งสองสามารถพบกันได้เป็นเท่าใด

หน้าที่ของคุณคือเขียนฟังก์ชัน

```
int meetup(station* s, station* t);
```

ที่คืนค่าเวลาที่น้อยที่สุดที่ทั้งสองสามารถพบกัน

รายละเอียดไฟล์ที่ให้

- `grader.cpp` : อ่านอินพุต สร้างโครงสร้างสถานี แล้วเรียก `meetup(s, t)`
- `meetup.h` : ประกาศโครงสร้าง `station` และฟังก์ชัน `meetup`
- `meetup.cpp` : ไฟล์ที่ผู้เข้าแข่งขันต้องเขียนและส่ง
- `sample.in` : ตัวอย่างอินพุต
- `runner.sh` : สคริปต์สำหรับคอมไพล์และรัน

ผู้เข้าแข่งขันไม่ต้องอ่านอินพุตเอง ใช้เฉพาะพอยน์เตอร์ `s` และ `t` ที่เกรดเดอร์ส่งให้

ข้อมูลเข้า

สำหรับไฟล์ `sample.in` ที่ให้มา มีรูปแบบข้อมูลดังนี้

1. บรรทัดแรกประกอบไปได้วยจำนวนเต็ม 3 จำนวน n จำนวนสถานีทั้งหมด, s ตำแหน่งของสถานีที่ฟรีเรนอยู่, และ t ตำแหน่งของสถานีที่ฮิมเมลอยู่
2. บรรทัดถัดมาประกอบไปด้วยจำนวนเต็ม $n-1$ จำนวน $t_1, ..., t_{n-1}$ แทนเวลาในการเดินทางระหว่างสองสถานี (สถานีที่ i และ $i+1$ จะใช้เวลาเดินทาง t_i

ผู้เข้าแข่งขันสามารถแก้ไขไฟล์ `sample.in` เพื่อทดสอบได้ตามใจ โดยไฟล์ `grader.cpp` จะทำการแปลงข้อมูลเข้านี้ให้อยู่รูปแบบที่เหมาะสมสำหรับฟังก์ชัน `meetup` (เป็นพอยน์เตอร์ s และ t)

ข้อมูลออก

สำหรับฟังก์ชัน `meetup` ที่ผู้เข้าแข่งขันต้องเขียนจะต้องคืนค่าจำนวนเต็ม เวลาน้อยที่สุดที่ทั้งสองสามารถพบกัน สำหรับไฟล์ `grader.cpp` จะเรียกฟังก์ชันนี้ด้วยค่าพารามิเตอร์ที่เหมาะสมแล้วพิมพ์ข้อความ `the minimum time for Frieren and Himmel to meet is x` เมื่อ x คือค่าที่ฟังก์ชัน `meetup` คืนค่ามา

เงื่อนไข

- จำนวนสถานีทั้งหมดมีค่าตั้งแต่ 2 ถึง 1,000
- ระยะเวลาในการเดินทางระหว่างสถานีที่ติดกันมีค่ามากกว่า 0 แต่ไม่เกิน 100,000

การรันโปรแกรม

- สำหรับระบบปฏิบัติการ Linux ให้ใช้คำสั่ง `sh runner.sh`
- สำหรับระบบปฏิบัติการ Windows ให้คอมไฟล์ด้วย
  `g++ -o grader grader.cpp meetup.cpp`
  จากนั้นรันโปรแกรมด้วยคำสั่ง
  `grader < sample.in`

การส่งไฟล์

ให้ส่งเฉพาะไฟล์ `meetup.cpp` ไปยังระบบตรวจเท่านั้น โดยภายในไฟล์ต้องมีการ `include` ไฟล์ `meetup.h` และต้องมีการนิยามฟังก์ชัน `int meetup(station *s, station *t)` ให้สมบูรณ์ ผู้เข้าแข่งขันสามารถประกาศตัวแปรเพิ่มเติมหรือ `include` ไลบรารีมาตรฐานอื่น ๆ ได้ตามความเหมาะสม

ตัวอย่างข้อมูลเข้าและข้อมูลออก

Input (Input File)	**Output**
8 1 4 2 4 5 9 8 1 2	6

ทางรถไฟมีทั้งหมด 8 สถานีและมีระยะเวลาเดินทางดังรูปด้านล่าง เริ่มต้นฟรีเรนอยู่สถานีซ้ายสุด ส่วนฮิมเมลอยู่สถานีที่ 4 นับจากซ้ายมือ ถ้าพวกเขานัดเจอกันที่สถานีที่ 2 ฟรีเรนจะใช้เวลาเดินทาง 2 หน่วย ฮิมเมลใช้เวลาเดินทาง 9 หน่วย พวกเขาจะเจอกันที่เวลา 9 หน่วย แต่ถ้าพวกเขานัดเจอกันที่สถานีที่ 3 ฟรีเรนจะใช้เวลาเดินทาง 6 หน่วย ฮิมเมลใช้เวลาเดินทาง 5 หน่วย ซึ่งจะเจอกันที่เวลา 6 หน่วย และเป็นเวลาที่น้อยที่สุดที่เป็นไปได้

